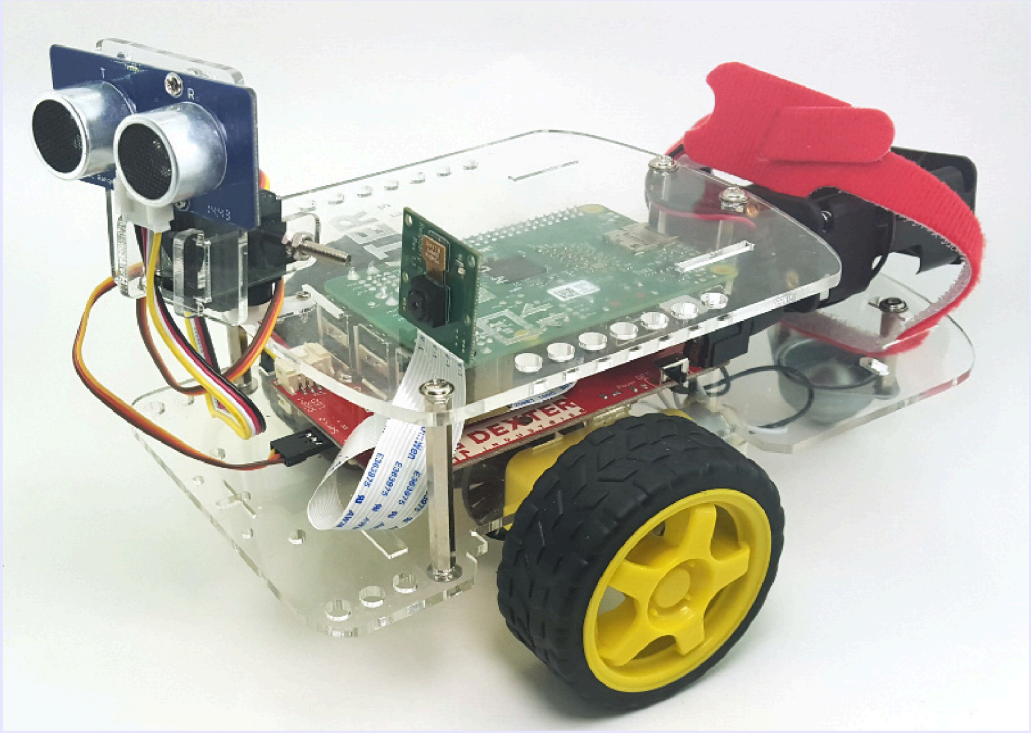
Rapport de Projet

LU2IN013

Ghrab Wassim 21306150
Su Florent 21217045
Chelali Rabah 21315151
Cherguia Abdelrrahmane 21314779
Fernane Said 2131508

Robot dexter



Sommaire

1. Introduction

2. Architecture Générale du Projet
   - 2.1. Organisation des dossiers
   - 2.2. Structure des modules

3. Couche Modèle (`MODEL/`)
   - 3.1. `robot.py` — Classe Robot
   - 3.2. `obstacle.py` — Classes Obstacle, Cercle, Rectangle, Ligne, Triangle
   - 3.3. `balise.py` — Classe Balise
   - 3.4. `environnement.py` — Classe Environnement

4. Couche Stratégies (`STRATEGY/`)
   - 4.1. Stratégies principales
   - 4.2. Fonctions d'initialisation

5. Couche Adaptation matériel/simulation (`ADAPTER/`)

6. Couche Détection (`ALGO_DETECTION/`)

7. Couche Affichage graphique (`VIEW/`)
   - 7.1. Version 2D — Tkinter (prototype)
   - 7.2. Version 3D — Ursina

8. Tests Unitaires

9. Points d'entrée et déroulement global
   - 9.1. `main3D.py` — Lancement de la simulation 3D
   - 9.2. `mainReel.py` — Lancement sur le robot réel

10. Conclusion

1. Introduction

Le Projet Robotique, réalisé dans le cadre de l'Unité d'Enseignement LU2IN013 à Sorbonne Université, avait pour objectif la conception et l'implémentation d'un robot autonome capable d'évoluer dans un environnement structuré, aussi bien en simulation qu'en mode réel.

Ce projet s'inscrit dans une démarche complète mêlant :

- la modélisation physique du robot et de son environnement (obstacles, balises),
- l'implémentation de stratégies de déplacement adaptables,
- l'utilisation de design patterns logiciels pour garantir une architecture souple, testable et évolutive,
- la gestion de la double compatibilité robot virtuel / réel (GoPiGo3),
- et l'affichage dynamique en 3D temps réel de la simulation.

Compétences mobilisées

- Programmation orientée objet (Python)
- Contrôle de robot différentiel
- Détection d'obstacles et suivi de balise
- Architecture logicielle modulaire (MVC)
- Application de patrons de conception (Strategy, Adapter)
- Développement de tests unitaires pour assurer la robustesse

Objectifs pédagogiques

- Concevoir un système robotique complet, de la modélisation au déploiement
- Séparer clairement les responsabilités entre le modèle, la vue et le contrôleur
- Permettre la réutilisabilité et l'extension future du code (stratégies multiples, mode exploration...)

2. Architecture Générale du Projet

Le projet a été structuré en couches logiques s'appuyant sur des design patterns reconnus comme MVC (Modèle-Vue-Contrôleur), ainsi que Strategy et Adapter.

Organisation des dossiers

src/ — fichier principal contenant l'ensemble des fichiers de code :

```
src/
├── FWSFR/              # Contient l'ensemble des modules fonctionnels du robot
├── TESTS/              # Contient les tests unitaires
├── main3D.py           # Fichier main pour la simulation 3D
└── mainReel.py         # Fichier main pour le robot réel
```

FWSFR/ — module principal (code source), organisé en 6 sous-modules, chacun avec un fichier `__init__.py` :

```
FWSFR/
├── MODEL/              # Entités du monde : robot, obstacles, balise, environnement
      ├── balise.py
      ├── environnement.py
      ├── obstacle.py
      └── robot.py
├── STRATEGY/           # Stratégies de déplacement et de comportement
      └── strategy.py
├── ADAPTER/            # Interface robot virtuel / réel (Adapter)
      ├── SCREENSHOT/
      └── adapter.py
├── ALGO_DETECTION/     # Algorithmes de détection
      ├── algo.py
└── VIEW/               # Affichage graphique (3D, via Ursina)
      └── affichage_3D.py
```

3. Couche Modèle (`MODEL/`)

Ce module représente l'environnement physique simulé dans lequel évolue le robot : son propre état (position, direction), les obstacles, les balises et les collisions. Il constitue le noyau des données manipulées par les vues et les adaptateurs. Les stratégies, quant à elles, interagissent exclusivement avec le modèle à travers l'adapter, ce qui garantit l'indépendance entre la logique de contrôle et l'implémentation concrète (simulation ou robot réel).

3.1. robot.py — Classe `Robot`

Implémente un robot différentiel virtuel capable d'avancer, tourner, percevoir son environnement, et détecter les collisions via des capteurs simulés.

Principaux attributs :

- `x, y` : position du centre du robot
- `direction` : angle d'orientation (radians)
- `vitesse_gauche`, `vitesse_droite` : vitesses des roues gauche/droite
- `distance_roues` : distance entre les roues
- `taille_robot` : taille du robot (pour calcul des collisions)
- `vitesse_max` : limite de vitesse
- `environnement` : référence à l'environnement courant

Méthodes clés :

- `mise_a_jour_robot(dt)` : met à jour la position et la direction du robot selon ses vitesses et la durée `dt`
- `appliquer_vitesse_gauche/droite(delta)` : applique un changement de vitesse, en respectant les limites
- `arreter_robot()` : arrête le robot (vitesses à zéro)
- `capteurdistance()` : simule deux capteurs (avant gauche/droite) pour détecter des obstacles devant le robot
- `points()` : calcule les sommets du triangle représentant le robot, ainsi que 3 points intermédiaires (utiles pour la détection de collision)

3.2. obstacle.py — Classes `Obstacle`, `Cercle`, `Rectangle`, `Ligne`, `Triangle`

- **Obstacle (abstraite)** : impose deux méthodes : `detecter_collision_point(point)` et `detecter_collision(robot)`

- **Cercle** : collision basée sur la distance au centre
- **Rectangle** : collision si le point est dans le rectangle
- **Ligne** : collision par projection du point sur le segment
- **Triangle** : collision basée sur le test du barycentre

3.3. balise.py — Classe `Balise`

- Modélise une balise visuelle avec : position, taille, rotation, hauteur, forme, et 4 couleurs en coin (simulation de repères visuels)
- Utilisée dans le suivi de balise (module stratégie + affichage)

3.4. environnement.py — Classe `Environnement`

- Représente l'arène (dimensions, obstacles, balises)
- Méthodes : `ajouter_obstacle()`, `ajouter_balise()`, `detecter_sorties()`, `gerer_collisions()`, `update()`
- Gestion de la sortie du cadre, repositionnement du robot, détection et gestion des collisions

4. Couche Stratégies (`STRATEGY/`)

Ce module implémente les différents comportements du robot sous forme de classes selon le design pattern Strategy, avec des méthodes standardisées :

- `execute()` : exécute l'action
- `__call__()` : initialise les paramètres
- `est_terminee()` : indique si la stratégie est terminée

Chaque stratégie utilise un objet `robot_adapter`, interface unifiée vers le robot réel ou virtuel.

4.1. Stratégies principales

- **StrategyAvancer** : avance sur une distance donnée
- **StrategyTourner** : tourne sur place d'un angle donné en degré
- **StrategyConditionnelle** : choisit dynamiquement entre deux stratégies selon une condition booléenne (ex : avancer si pas d'obstacle, sinon tourner)
- **StrategySequentielle** : enchaîne plusieurs stratégies dans un ordre donné (utile pour tracer un carré, etc.)
- **StrategySuivreBalise** : fait suivre une balise visuelle au robot, en analysant la position de la balise dans l'image (gauche/centre/droite) et ajuste les vitesses
- **StrategyUnDeuxTroisSoleil** : variante qui stoppe dès que la balise disparaît et s'arrête définitivement si la balise est proche (simulation d'un jeu "1, 2, 3 soleil")

4.2. Fonctions d'initialisation

- `initialiser_strategies(robot_adapter)` : crée et retourne une séquence de stratégies prêtes à être enchaînées
- `verif_sequence(sequence)` : exécute la stratégie courante si elle n'est pas terminée

5. Couche Adaptation matériel/simulation (`ADAPTER/`)

Ce module gère l'abstraction du robot, permettant au reste du programme de fonctionner indifféremment sur un robot simulé ou physique. Il repose sur le design pattern Adapter, avec :

- **RobotAdapter** (abstraite) : définit l'interface commune (méthodes pour moteurs, distance, image, etc.)
- **RobotAdapterSimulation** : adaptation pour le robot virtuel, contrôle la simulation et capture les images (screenshots 3D)
- **RobotAdapterReel** : adaptation pour le GoPiGo3, contrôle les moteurs réels et récupère les images/capteurs du robot

6. Couche Détection/ALGO_DETECTION

Ce module réalise l'analyse d'image basée sur la couleur pour localiser une balise multicolore, vue par la caméra du robot (ou un screenshot en simulation), en utilisant OpenCV.

- **generer_masque_balise(image_rgb)** : prend une image RGB, convertit en HSV, masque les couleurs caractéristiques, filtre le bruit (morphologie)

- **position_balise_dans_image(masque)** : estime la position de la balise ("gauche", "centre", "droite", ou None) en calculant le centroïde de la zone blanche du masque

Ces fonctions sont intégrées dans les stratégies de suivi de balise.

7. Couche Affichage graphique (`VIEW/`)

7.1. Version 2D — Tkinter (prototype)

- Affichage simple du robot, obstacles, trajectoire en 2D

- Utile pour le débogage et la validation rapide des stratégies

7.2. Version 3D — Ursina

- Visualisation en temps réel : robot, obstacles, balises, évolution du robot

- **SimulationView3D** : classe principale, gère la fenêtre, l'affichage, la caméra libre ou embarquée, les infos dynamiques

- **switch()** : bascule entre caméra libre et vue robot (TAB)

- **render()** : capture un screenshot (utile pour la détection de balise en simulation)

- **afficher_robot()**, **afficher_obstacles()**, **afficher_balises()** : actualisent la position/forme/couleur des objets dans la scène

8. Tests Unitaires

Un répertoire de tests automatisés a été conçu pour vérifier :

- les déplacements et collisions du robot (simulation uniquement),
- la validité des méthodes d'environnement et obstacles,
- la robustesse de chaque composant (robot, environnement, obstacles...)

9. Points d'entrée et déroulement global

Le projet propose plusieurs scénarios d'utilisation, chacun avec son propre fichier principal (main) :

• `main3D.py` — Lancement de la simulation 3D

Ce fichier est le point d'entrée pour lancer la simulation complète du robot dans un environnement 3D, avec visualisation en temps réel via Ursina.

Fonctionnement :

- Initialise l'environnement, le robot simulé, la vue 3D, et les stratégies de déplacement.
- Lance une boucle principale dans laquelle :
  - Le temps est mis à jour à chaque itération,
  - Les stratégies sont exécutées via la séquence courante,
  - L'environnement, le robot et l'affichage 3D sont mis à jour,
  - Un délai court régule le nombre de frames par seconde.

• `mainReel.py` — Lancement sur le robot réel

Ce fichier est le point d'entrée pour piloter le robot physique (GoPiGo3), avec les mêmes stratégies de déplacement que la simulation.

Fonctionnement :

- Met à jour la mesure du temps,
- Exécute la stratégie courante (par l'adapter, qui communique directement avec le matériel du robot),
- Régule la fréquence de la boucle (pour éviter de surcharger le matériel).

10. Conclusion

Ce projet de robotique nous a permis de concevoir, programmer et tester un robot autonome capable d'évoluer en simulation comme en conditions réelles. Grâce à une architecture modulaire claire et l'utilisation de plusieurs design patterns (Strategy et Adapter), le système est à la fois robuste, maintenable et flexible.

Le développement a été progressif :

- Première version 2D avec Tkinter pour valider les bases (mouvement, collisions, tracé de figures)
- Version 3D avec Ursina pour un rendu immersif, une simulation dynamique et l'intégration de la vision par ordinateur (suivi de balise)
- Passage au robot réel (GoPiGo3), avec réutilisation directe des stratégies grâce à l'adapter

Le projet a été très intéressant à réaliser, car il nous a permis de mettre en pratique des notions importantes comme les design patterns (Strategy, Adapter), la programmation orientée objet en Python, et l'organisation modulaire d'un code complexe. Travailler en groupe nous a aussi appris à mieux nous organiser, notamment avec des outils comme Trello pour le suivi des tâches et GitHub pour la gestion du code.

Finalement, ce projet nous a permis d'acquérir de nouvelles compétences en robotique, en développement logiciel et en travail d'équipe, tout en nous donnant une vue d'ensemble sur ce que peut être un vrai projet technique.